Manual Técnico del Sistema BUNI

Plataforma de Gestión de Bienestar Universitario

Versión: 1.0

Fecha: Febrero 2025

MANUAL TÉCNICO DEL SISTEMA BUNI

Plataforma de Gestión de Bienestar Universitario

Información del Documento

Campo	Detalle
Proyecto	BUNI - Bienestar Universitario
Versión	1.0
Fecha	Febrero 2025
Institución	Universidad del Sinú Elías Bechara Zainum

Personal Responsable

Nombre	Rol	Responsabilidad	Contacto
Sebastian Garcés Gómez	Gestión completa del equipo, desarrollo, diseño y control de calidad	Dirección general y toma de decisiones estratégicas	sebastiangarces152@gmail.com
Juan David López	Gestión completa del equipo, desarrollo, diseño y control de calidad	Garantizar la calidad del software y detección de errores	lopezpalominojuand@gmail.com
Jonathan Stick Tena Figueroa	Ingeniero Validador	Validación técnica del proyecto	-

---

ÍNDICE

1. Introducción al Sistema
    - 1.1 Descripción General
    - 1.2 Arquitectura del Sistema
    - 1.3 Especificaciones Técnicas
    - 1.4 Funcionalidades Principales
2. Instalación y Configuración
    - 2.1 Preparación del Entorno
    - 2.2 Configuración del Backend
    - 2.3 Configuración del Frontend
    - 2.4 Configuración de Base de Datos
3. Administración del Sistema
    - 3.1 Acceso Inicial
    - 3.2 Gestión de Usuarios
    - 3.3 Administración de Eventos
    - 3.4 Gestión de Formularios
4. Operación Diaria
    - 4.1 Dashboard y Monitoreo
    - 4.2 Gestión de Eventos
    - 4.3 Préstamo de Escenarios
5. Mantenimiento y Respaldos
    - 5.1 Respaldos de Base de Datos
    - 5.2 Monitoreo de Logs
    - 5.3 Procedimientos de Actualización
6. Solución de Problemas
    - 6.1 Diagnóstico de Problemas Comunes
    - 6.2 Procedimientos de Emergencia
7. Seguridad y Auditoría
    - 7.1 Medidas de Seguridad
    - 7.2 Logs de Auditoría
8. Referencias y Anexos
    - 8.1 Comandos de Referencia
    - 8.2 Estructura de Archivos

---

1. Introducción al Sistema

1.1 Descripción General

El sistema BUNI (Bienestar Universitario) es una aplicación web desarrollada específicamente para automatizar las tareas del área de Bienestar Universitario de la Universidad del Sinú. La plataforma centraliza la gestión de eventos deportivos y culturales, el registro de participantes, la administración de escenarios recreativos y la generación de reportes en tiempo real.

Problemática Resuelta

Problema Manual Actual	Solución Automatizada BUNI
Carga operativa significativa para coordinadores	Sistema centralizado de gestión
Errores de transcripción	Validación automática de datos
Falta de información en tiempo real	Dashboard con métricas actualizadas
Gestión descentralizada de eventos	Plataforma única integrada
Procesos manuales de inscripción	Códigos QR automáticos
Dificultad en generación de reportes	Exportación automática a Excel

1.2 Arquitectura del Sistema

El sistema BUNI implementa una **arquitectura de tres capas** que separa las responsabilidades para garantizar escalabilidad y mantenibilidad:

Arquitectura Backend

Capa	Responsabilidad	Tecnologías
Controladores	Manejo de solicitudes HTTP y respuestas	Express.js Routes
Servicios	Lógica de negocio y validaciones	JavaScript ES6+
Modelos	Interacción con base de datos	MySQL queries directos

Stack Tecnológico Completo

Componente	Tecnología	Versión	Función Específica
Frontend	Next.js	14.x	Server-Side Rendering y optimización
Frontend	React	18.x	Interfaz de usuario reactiva
Frontend	TypeScript	5.x	Tipado estático y desarrollo seguro
Frontend	Tailwind CSS	3.x	Diseño responsive y rápido
Frontend	Recharts	2.x	Gráficos y visualización de datos
Frontend	ExcelJS	4.x	Importación/exportación de Excel
Frontend	Lucide React	-	Biblioteca de iconos
Frontend	Radix UI	-	Componentes accesibles
Backend	Node.js	18.x	Runtime JavaScript del servidor
Backend	Express.js	4.x	Framework web y API REST
Backend	bcryptjs	-	Encriptación de contraseñas
Backend	JWT	-	Autenticación y autorización
Comunicación	Socket.IO	4.x	Actualizaciones en tiempo real
Base de Datos	MySQL	5.7+	Almacenamiento persistente

1.3 Especificaciones Técnicas

Requisitos de Hardware

Componente	Mínimo	Recomendado	Notas
Servidor	4GB RAM, 2 CPU cores	8GB RAM, 4 CPU cores	Para hasta 100 usuarios concurrentes
Almacenamiento	50GB disponibles	100GB disponibles	Incluye logs y respaldos
Clientes	Procesador 1.66GHz, 4GB RAM	Procesador 2.4GHz, 8GB RAM	Navegador moderno requerido
Red	Conexión estable a internet	Ancho de banda mínimo 10 Mbps	Para actualizaciones en tiempo real

Requisitos de Software

Sistema	Versión Mínima	Versión Recomendada	Compatibilidad
Windows	Windows 7	Windows 10/11	Soporte completo
Linux	Ubuntu 18.04	Ubuntu 20.04+	Preferido para producción
macOS	macOS 10.14	macOS 12+	Desarrollo y testing
Node.js	18.x	20.x LTS	Requerido para backend
MySQL	5.7	8.0	InnoDB engine requerido

Navegadores Soportados y Características

Navegador	Versión Mínima	Características Soportadas
Google Chrome	90+	WebSockets, PWA, QR Scanner

1.4 Funcionalidades Principales

Sistema de Autenticación

Característica	Descripción	Tecnología
Login Seguro	Autenticación por email y contraseña	JWT + bcrypt
Control de Roles	Permisos granulares por funcionalidad	Sistema de roles personalizado
Recuperación de Contraseña	Códigos de recuperación por email	SMTP + códigos temporales
Sesiones Seguras	Tokens con expiración automática	JWT con refresh tokens

Dashboard Administrativo

Métrica	Visualización	Actualización	Función
Eventos Activos	Contador dinámico	Tiempo real	Monitoreo de actividades
Participantes	Gráfico de barras	Automática	Análisis de participación
Asistencia	Gráfico circular	Por evento	Control de presencia
Usuarios Activos	Lista actualizada	Cada 5 minutos	Gestión de sesiones

---

Gestión Avanzada de Eventos

Funcionalidad	Descripción	Beneficio
Códigos QR Automáticos	Generación automática para inscripción y asistencia	Eliminación de procesos manuales
Formularios Dinámicos	Campos personalizables según tipo de evento	Flexibilidad en recolección de datos
Gestión de Cupos	Control automático de límites de participantes	Prevención de sobrecupo
Notificaciones	Alertas automáticas por email	Comunicación eficiente
Reportes Automáticos	Generación post-evento con estadísticas	Análisis de efectividad

Sistema de Formularios Dinámicos

Tipo de Campo	Validación	Uso Específico	Ejemplo
Texto Simple	Longitud máxima, caracteres especiales	Nombres, descripciones	Nombre completo
Números	Rango, formato	Códigos estudiantiles, teléfonos	Código: 201912345
Fechas	Formato, rango válido	Fechas de nacimiento	DD/MM/AAAA
Selección Múltiple	Lista predefinida	Programas académicos	Ingeniería, Medicina
Casillas	Booleano	Términos y condiciones	Acepto políticas
Calificaciones	Escala numérica	Evaluaciones de eventos	1-5 estrellas
Entre otros	**Entre otros**	**Entre otros**	**Entre otros**

2. Instalación y Configuración

2.1 Preparación del Entorno

Instalación de Node.js

Paso	Acción	Comando/URL
1	Descargar Node.js LTS	https://nodejs.org
2	Ejecutar instalador	Seguir wizard de instalación
3	Verificar instalación	`node --version && npm --version`
4	Configurar npm global	`npm config set prefix ~/.npm-global`

Configuración de MySQL

```sql
-- Crear base de datos principal
CREATE DATABASE buni CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Crear usuario específico para BUNI
CREATE USER 'buni_user'@'localhost' IDENTIFIED BY 'secure_password_2025!';

-- Otorgar permisos completos
GRANT ALL PRIVILEGES ON buni.* TO 'buni_user'@'localhost';

-- Aplicar cambios
FLUSH PRIVILEGES;

-- Verificar creación
SHOW DATABASES;
SELECT User, Host FROM mysql.user WHERE User = 'buni_user';
```

2.2 Configuración del Backend

Variables de Entorno Completas

Crear archivo `.env` en el directorio `api-buni`:

```
PORT="4000"                  # Puerto del servidor

# Database
DB_HOST="localhost"         # Host de la base de datos
DB_USER="root"              # Usuario de la base de datos
DB_PASSWORD=""              # Contraseña de la base de datos
DB_NAME="buni"              # Nombre de la base de datos
DB_PORT="3306"              # Puerto de la base de datos

# CORS
CORS_ORIGINS="http://localhost:3000"  # Origenes permitidos para CORS

# JWT
JWT_SECRET="your_secret_key"  # Clave secreta para JWT

# Credentials (usuario administrador por defecto)
ADMIN_NAME="Administrador"
ADMIN_EMAIL="admin@example.com"
ADMIN_PASSWORD="admin123"

# Rate limit
```

```
RATE_LIMIT_WINDOW_MINUTES=1  # Ventana de tiempo para rate limiting en minutos
RATE_LIMIT_MAX=200           # Máximo de peticiones en la ventana de tiempo

# Transport Email
SMTP_HOST=smtp.gmail.com     # Host del servidor SMTP
SMTP_PORT=587                # Puerto del servidor SMTP
SMTP_USER=example@gmail.com  # Usuario del servidor SMTP
SMTP_PASS=your_app_password  # Contraseña de aplicación
```

Estructura de Directorios Backend

```
api-buni/
├── src/
│   ├── controllers/          # Controladores HTTP
│   │   ├── auth.controller.js
│   │   ├── users.controller.js
│   │   ├── events.controller.js
│   │   └── forms.controller.js
│   │   └── Entre otros..
│   ├── services/             # Lógica de negocio
│   │   ├── auth.service.js
│   │   ├── users.service.js
│   │   ├── events.service.js
│   │   └── email.service.js
│   │   └── Entre otros..
│   ├── models/               # Modelos de datos
│   │   ├── user.model.js
│   │   ├── event.model.js
│   │   └── form.model.js
│   │   └── Entre otros..
│   ├── middleware/           # Middlewares
│   │   ├── auth.middleware.js
│   │   ├── event.middleware.js
│   │   └── user.middleware.js
│   │   └── Entre otros..
│   ├── routes/               # Definición de rutas
│   │   ├── auth.routes.js
│   │   ├── users.routes.js
│   │   └── events.routes.js
│   │   └── Entre otros..
│   ├── lib/                  # Utilidades
│   │   ├── Utils.js
│   │   ├── EmailManager.js
│   │   └── CodeManager.js
│   │   └── Entre otros..
│   └── config/               # Configuraciones
│       ├── database.js
├── .env                      # Variables de entorno
├── package.json
└── index.js                  # Punto de entrada
```

Comandos de Instalación Backend

```bash
# Clonar o descargar el repositorio
cd api-buni
```

```
# Instalar dependencias de producción
npm install --production

# Para desarrollo (incluye dependencias de dev)
npm install

# Crear directorio de logs
mkdir -p logs

# Verificar configuración
npm run config-check

# Iniciar en modo desarrollo
npm run dev

# Iniciar en modo producción
npm start
```

2.3 Configuración del Frontend

Variables de Entorno Frontend

Crear archivo `.env.local` en el directorio `buni-frontend`:

```
# ================================
# CONFIGURACIÓN DE API
# ================================
NEXT_PUBLIC_WEBSOCKET_URL=ws://localhost:4000
NEXT_PUBLIC_API_URL=http://localhost:4000
```

Comandos de Instalación Frontend

```
# Navegar al directorio del frontend
cd buni-frontend

# Instalar dependencias
npm install

# Generar archivos de tipos TypeScript
npm run type-check

# Compilar para producción
npm run build

# Iniciar en modo desarrollo
npm run dev

# Iniciar en modo producción
npm start

# Verificar lint y formateo
```

```
npm run lint
npm run format
```

2.4 Configuración de Base de Datos

Esquema de Base de Datos Completo

Tabla: Roles

```sql
CREATE TABLE roles (
    id VARCHAR(255) PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,
    permissions JSON NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tabla: Usuarios

```sql
CREATE TABLE users (
    id VARCHAR(255) PRIMARY KEY,
    role_id VARCHAR(255) NULL,
    name VARCHAR(255) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE SET NULL
);
```

Tabla: Eventos

```sql
CREATE TABLE events (
    id VARCHAR(255) PRIMARY KEY,
    nombre VARCHAR(255) NOT NULL,
    organizador VARCHAR(255) NOT NULL,
    scenery JSON NOT NULL,
    programs JSON NOT NULL,
    cupos VARCHAR(255) NOT NULL,
    horarioInicio DATETIME NOT NULL,
    horarioFin DATETIME NOT NULL,
    state VARCHAR(255) NOT NULL,
    formAssists JSON NOT NULL,
    formInscriptions JSON NOT NULL,
    assists JSON NOT NULL,
    inscriptions JSON NOT NULL,
    formConfig JSON NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tabla: Escenarios

```sql
CREATE TABLE sceneries (
    id VARCHAR(255) PRIMARY KEY,
    name VARCHAR(255) UNIQUE NOT NULL,
    state VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tabla: Programas

```sql
CREATE TABLE programs (
    id VARCHAR(255) PRIMARY KEY,
    name VARCHAR(255) UNIQUE NOT NULL,
    state VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tabla: Formularios

```sql
CREATE TABLE forms (
    id VARCHAR(255) PRIMARY KEY,
    name VARCHAR(255) NOT NULL UNIQUE,
    description VARCHAR(255) NOT NULL,
    fields JSON NOT NULL,
    state VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Triggers de Base de Datos

```sql
-- Trigger para configuración inicial de eventos
DELIMITER //
CREATE TRIGGER before_event_insert
BEFORE INSERT ON events
FOR EACH ROW
BEGIN
    IF NEW.formConfig IS NULL OR JSON_LENGTH(NEW.formConfig) = 0 THEN
        SET NEW.formConfig = JSON_OBJECT(
            'inscriptions', JSON_OBJECT(
                'enabled', false,
                'startDate', '',
                'endDate', ''
            ),
            'assists', JSON_OBJECT(
                'enabled', false,
                'startDate', '',
                'endDate', ''
            )
        );
```

```
    END IF;
END //
DELIMITER ;
```

Datos Iniciales del Sistema

```
-- Insertar roles predeterminados
INSERT INTO roles (id, name, permissions) VALUES 
('role-superadmin', 'SuperAdmin', JSON_OBJECT(
    'users', JSON_ARRAY('create', 'read', 'update', 'delete'),
    'events', JSON_ARRAY('create', 'read', 'update', 'delete'),
    'forms', JSON_ARRAY('create', 'read', 'update', 'delete'),
    'programs', JSON_ARRAY('create', 'read', 'update', 'delete'),
```

Problemática Resuelta

Problema Manual Actual	Solución Automatizada BUNI
Carga operativa significativa para coordinadores	Sistema centralizado de gestión
Errores de transcripción	Validación automática de datos
Falta de información en tiempo real	Dashboard con métricas actualizadas
Gestión descentralizada de eventos	Plataforma única integrada

---

3. Administración del Sistema

3.1 Acceso Inicial

Credenciales por Defecto

Campo	Valor
URL	http://localhost:3000
Usuario	admin@unisinu.edu.co
Contraseña	admin123

> ⚠️ **IMPORTANTE:** Cambiar estas credenciales inmediatamente después del primer acceso.

3.2 Gestión de Usuarios

Roles del Sistema

Rol	Permisos	Funciones Principales
SuperAdmin	Acceso completo	Gestión total del sistema

Rol	Permisos	Funciones Principales
Administrador	Gestión de eventos y usuarios	Coordinación general
Coordinador	Eventos de su área	Gestión específica por dependencia
Usuario	Participación en eventos	Inscripciones y consultas

Proceso de Creación de Usuarios

1. **Acceso**: Usuarios → Listado de Usuarios
2. **Agregar**: Clic en botón "+ Agregar"
3. **Completar datos**:
    - Nombre completo
    - Correo electrónico institucional
    - Contraseña temporal
    - Rol asignado

3.3 Administración de Eventos

Estructura de Eventos

Campo	Tipo	Descripción
Nombre	Texto	Denominación del evento
Organizador	Selección	Área responsable
Participantes	Lista	Personas registradas
Fecha	DateTime	Fecha y hora del evento
Área	Enumeration	Coordinación encargada
Cupos	Número	Límite de participantes

Códigos QR Automáticos

El sistema genera automáticamente:

- **QR de Inscripción**: Para registro de participantes
- **QR de Asistencia**: Para control de presencia

3.4 Gestión de Formularios

Tipos de Campos Disponibles

Tipo de Campo	Uso	Validación
Texto Simple	Nombres, descripciones	Longitud máxima
Números	Códigos, identificaciones	Formato numérico

Tipo de Campo	Uso	Validación
Fechas	Fechas de nacimiento	Formato DD/MM/AAAA
Selección Múltiple	Programas, opciones	Lista predefinida
Casillas de Verificación	Aceptación de términos	Booleano
Entre otros	**Entre otros**	**Entre otros**

---

4. Operación Diaria

4.1 Dashboard y Monitoreo

Métricas Principales

Métrica	Descripción	Actualización
Eventos Activos	Eventos en curso o próximos	Tiempo real
Participantes Registrados	Total de inscripciones	Automática
Préstamos Pendientes	Escenarios reservados	En vivo
Usuarios Activos	Sesiones vigentes	Cada 5 minutos

Indicadores de Salud del Sistema

```
# Verificar estado del backend
curl http://localhost:4000/health

# Verificar procesos
ps aux | grep node

# Verificar memoria
free -h
```

4.2 Gestión de Eventos

Flujo de Trabajo Durante un Evento

Fase	Acciones	Responsable
Pre-evento	Verificar inscripciones, preparar QR	Coordinador
Durante	Control de asistencia, monitoreo	Administrador
Post-evento	Generar reportes, exportar datos	Sistema/Coordinador

---

5. Mantenimiento y Respaldos

5.1 Respaldos de Base de Datos

Programación de Respaldos

Frecuencia	Comando	Retención
Diario	`mysqldump -u buni_user -p buni > backup_daily.sql`	7 días
Semanal	`mysqldump -u buni_user -p buni > backup_weekly.sql`	4 semanas
Mensual	`mysqldump -u buni_user -p buni > backup_monthly.sql`	12 meses

Script de Respaldo Automático

```
#!/bin/bash
# backup_buni.sh
DATE=$(date +%Y%m%d_%H%M%S)
mysqldump -u buni_user -p buni > /backup/buni_$DATE.sql
find /backup -name "buni_*.sql" -mtime +7 -delete
```

5.2 Monitoreo de Logs

Ubicación de Logs

Componente	Ubicación	Propósito
Backend	`api-buni/logs/`	Errores de aplicación
Sistema	`/var/log/` (Linux)	Errores del sistema
Base de Datos	`/var/log/mysql/`	Logs de MySQL

5.3 Procedimientos de Actualización

Lista de Verificación Pre-Actualización

- Crear respaldo completo de la base de datos
- Verificar espacio en disco disponible
- Notificar a usuarios sobre mantenimiento
- Documentar versión actual

---

6. Solución de Problemas

6.1 Diagnóstico de Problemas Comunes

Matriz de Problemas y Soluciones

Problema	Posible Causa	Solución
Error de conexión a BD	MySQL detenido	`systemctl start mysql`
Sistema no responde	Memoria insuficiente	Reiniciar servicios
Formularios no se envían	Sesión expirada	Refrescar página
Códigos QR no funcionan	Error en generación	Verificar logs del backend

6.2 Procedimientos de Emergencia

Restauración Rápida

1. **Detener servicios**
2. **Restaurar base de datos**: `mysql -u buni_user -p buni < backup.sql`
3. **Reiniciar servicios**
4. **Verificar funcionamiento**

Contacto de Emergencia

En caso de problemas críticos, contactar:

- **Sebastian Garcés**: sebastiangarces152@gmail.com
- **Juan David López**: lopezpalominojuand@gmail.com

---

7. Seguridad y Auditoría

7.1 Medidas de Seguridad

Políticas de Contraseñas

Requisito	Especificación
Longitud mínima	8 caracteres
Complejidad	Letras, números y símbolos
Rotación	Cada 90 días
Historial	No reutilizar últimas 5

7.2 Logs de Auditoría

Eventos Registrados Automáticamente

- Inicios y cierres de sesión
- Creación, modificación y eliminación de eventos
- Cambios en permisos de usuarios

- Exportación de datos sensibles

---

8. Referencias y Anexos

8.1 Comandos de Referencia

```
# Gestión de procesos con PM2
pm2 status                    # Ver estado de servicios
pm2 logs buni-api             # Ver logs en tiempo real
pm2 restart buni-api          # Reiniciar backend
pm2 monit                     # Monitor de recursos

# Base de datos
mysqldump -u buni_user -p buni > backup.sql  # Crear respaldo
mysql -u buni_user -p buni < backup.sql      # Restaurar respaldo
```

8.2 Estructura de Archivos

```
/proyecto-buni/
├── api-buni/
│   ├── .env                  # Configuración del backend
│   ├── src/                  # Código fuente
│   └── logs/                 # Logs del sistema
├── buni-frontend/
│   ├── .env.local            # Configuración del frontend
│   └── .next/                # Archivos compilados
└── backups/                  # Respaldos de base de datos
```

Manual Técnico - Sistema BUNI v1.0 *Universidad del Sinú Elías Bechara Zainum*
Febrero 2025